

Predicting Depth Maps from SPAD Images

Shahid Mohammad Mulla

Indian Institute of Technology Madras

April 30, 2025

- 1 Models used for Experiments
 - Overview
 - Vision Transformers (ViT) with CNN Stem
- 2 Best performing model
 - Pre Processing for U-Net Model
 - U-Net + CBAM Architecture
 - CBAM Module
 - Loss Function
- 3 Final results
- 4 Attempted Improvements
- 5 Leaderboard Performance

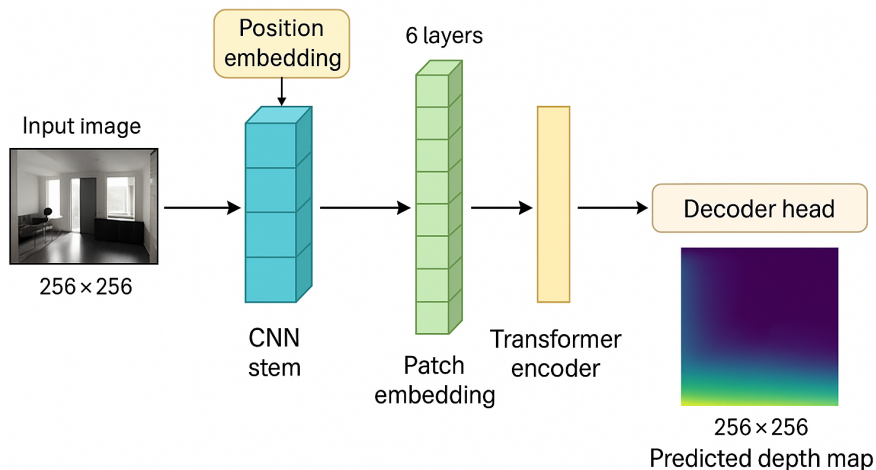
Model Experiments

- **Baseline:** U-Net trained directly on sparse SPAD binary image
- **Improved:** U-Net + CBAM with 3-channel input (SPAD + 2 pseudo-channels)
- **Architectural Modifications:**
 - Additional CBAM added to bottleneck
 - Residual connections within blocks
 - Leaky ReLU instead of ReLU for smoother gradient flow
- **Explorations:**
 - Vision Transformers (ViT) with CNN Stem —
 - Initial experiments with plain ViT were not working greatly
 - Introduced a lightweight CNN encoder before transformer blocks
 - Helped capture local patterns in sparse SPAD images before global reasoning
 - Resulted in sharper and more stable depth predictions

Architecture Overview

- **CNN Stem:** Initial feature extraction using 3-layer CNN
 - Helps retain local structure and fine details
 - Outputs 128 channels from input SPAD + pseudo-channels
- **Patch Embedding:** CNN features split into 16×16 patches
 - Each patch linearly projected to a 768-d embedding
- **Transformer Encoder:** 6-layer ViT block with 8 attention heads
 - Self-attention for capturing global spatial dependencies
- **Decoder Head:** Simple linear decoder reshapes tokens back to full resolution
 - Output: Single-channel depth map of size 256×256

Vision Transformer (ViT) with CNN Stem



- **Why we used CNN Stem:**

- The normal ViT model was not doing well on our very sparse SPAD input
- CNN helps the model understand local patterns like edges and shapes early on
- This gave the ViT better features to work with

- **Advantages over Standard ViT:**

- Training was smoother and more stable
- More accurate depth structure in highly textured regions
- It handled the sparsity of the data better

- **Drawbacks:**

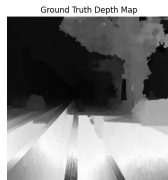
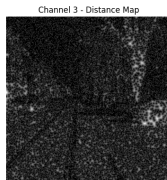
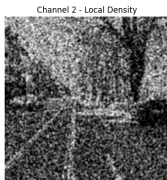
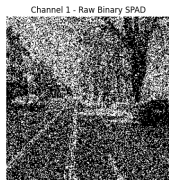
- Increased computational load due to added CNN layers
- Decoder was too simple — struggled to recover fine-grained spatial details
- Did not outperform our best CNN-based model (U-Net + CBAM)

Pre Processing for U-Net Model

Preprocessing

- Added two additional input channels to supplement sparse binary SPAD image:
 - **Local Density Map:** 3×3 average (acts like a soft edge detector)
 - **Distance Map:** Distance to nearest zero pixel (reveals structure)

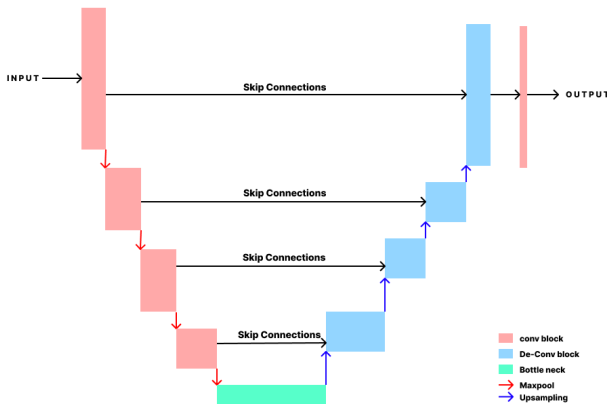
Visual Comparison:



Left to Right: Raw SPAD, Density Map, Distance Map, Ground Truth Depth

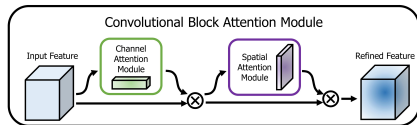
U-Net + CBAM Architecture

- Each block consists of the following operations:
 - Convolution $\rightarrow$ ReLU $\rightarrow$ Convolution $\rightarrow$ ReLU $\rightarrow$ CBAM
- The encoder and decoder blocks follow this structure, with skip connections between corresponding encoder and decoder layers.



CBAM: Convolutional Block Attention Module

- CBAM sequentially applies **Channel** and **Spatial** attention to enhance feature maps.
- Lightweight and can be integrated into existing CNNs with minimal overhead.



Reference: Woo et al., "CBAM: Convolutional Block Attention Module", ECCV 2018. <https://arxiv.org/abs/1807.06521v2>

BerHu Loss Function

The BerHu loss function is a combination of the L1 loss and a smooth approximation of the L2 loss. It is defined as:

$$\mathcal{L}_{\text{BerHu}}(\text{pred}, \text{target}) = \begin{cases} |\text{pred} - \text{target}| & \text{if } |\text{pred} - \text{target}| \leq c \\ \frac{(\text{pred} - \text{target})^2 + c^2}{2c} & \text{if } |\text{pred} - \text{target}| > c \end{cases} \quad (1)$$

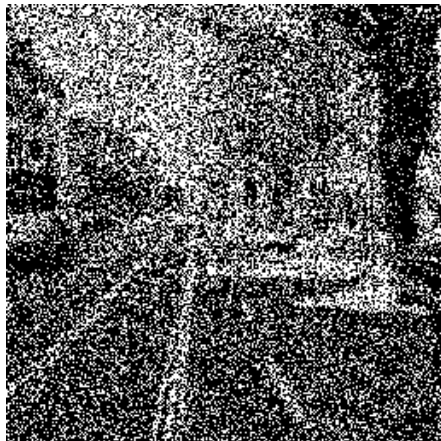
where $c = 0.2 \times \max(|\text{pred} - \text{target}|)$.

Small differences $\rightarrow$ usually noise $\rightarrow$ use L1 $\rightarrow$ penalize less

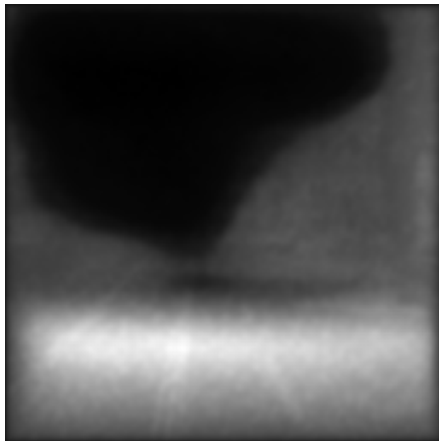
Large differences $\rightarrow$ real errors $\rightarrow$ use L2 $\rightarrow$ penalize more

Final Results

Input

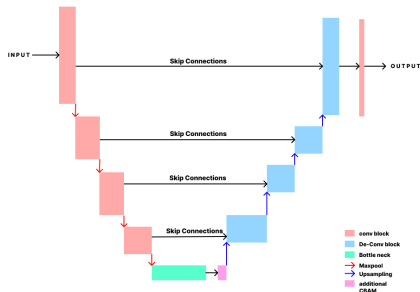


output

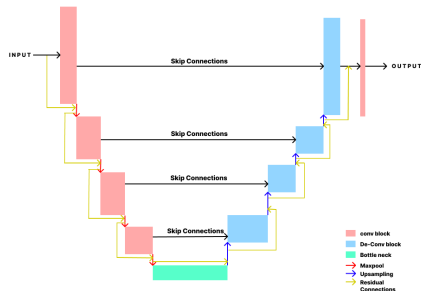


Additional Attempts on Best Model

CBAM at Bottleneck + other improvements



Residual connections



Public Leaderboard

Score: *0.206188*

Rank: *11*

Private Leaderboard

Score: *0.209318*

Rank: *12*